



INSTITUTO POLITÉCNICO NACIONAL



# **INSTITUTO POLITÉCNICO NACIONAL**

**ESCUELA SUPERIOR DE CÓMPUTO**

**SISTEMAS EN CHIP**

**Reporte de la práctica 10**

**“Proyecto 1”**

**Grupo: 6CM1**

## **Integrantes**

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

## **Profesor**

**Aguilar Sánchez Fernando**

**Fecha de entrega: 20 de marzo de 2025**

# INTRODUCCIÓN

## Motorreductores

Un motorreductor es la combinación de un motor eléctrico (generalmente de corriente continua o DC) y un tren de engranajes acoplado.

### ¿Por qué se utilizan?

Los motores de DC pequeños giran muy rápido (miles de RPM) pero tienen muy poca fuerza (par motor o torque). Si intentaras mover una rueda directamente con el eje del motor, este se detendría al menor roce. El motorreductor soluciona esto:

- **Reducción de velocidad:** Los engranajes reducen las revoluciones de salida.
- **Aumento de Torque:** Por leyes de la física, al reducir la velocidad mediante engranajes, la fuerza de giro aumenta proporcionalmente.
- **Relación de Transmisión (Ratio):** Se expresa como 1:48, 1:120, etc. Un ratio de 1:100 significa que el motor interno debe girar 100 veces para que la rueda exterior gire una sola vez.

## Infrarrojos (IR)

Los sistemas infrarrojos funcionan en el espectro electromagnético justo por debajo de la luz roja visible (700 nm a 1 mm). En electrónica, se usan principalmente de dos formas:

### Sensores de Barrera (Ranurados)

Es un pequeño bloque con forma de "U". De un lado hay un emisor IR y del otro un fototransistor.

- **Uso en motores:** Se usan junto con un disco encoder (un disco con ranuras) acoplado al eje del motor. Cuando el motor gira, el disco interrumpe el haz de luz.
- **Medición:** Contando estas interrupciones con el ATmega8535, se puede saber exactamente a qué velocidad gira la rueda o qué distancia ha recorrido (Odometría).

### Sensores de Proximidad

Contienen el emisor y el receptor apuntando en la misma dirección.

- **Detección de objetos:** Si un objeto se acerca, la luz IR rebota y el receptor la detecta.
- **Seguidores de línea:** Se basan en que el color blanco refleja la luz IR, mientras que el color negro la absorbe. El sensor detecta esta diferencia de voltaje y el microcontrolador decide hacia dónde girar.

## Integración: El Puente H

Es importante notar que no se puede conectar un motorreductor directamente a los pines de un microcontrolador porque el motor consume mucha corriente y quemaría el chip.

- Se utiliza un Puente H (como el chip L293D o L298N).
- Este actúa como una etapa de potencia que recibe señales pequeñas del microcontrolador y permite controlar tanto el encendido/apagado como el sentido de giro del motorreductor.

## DESARROLLO

### Circuito en hecho en Proteus

Para el desarrollo de la práctica 1, el circuito a armar es el siguiente:

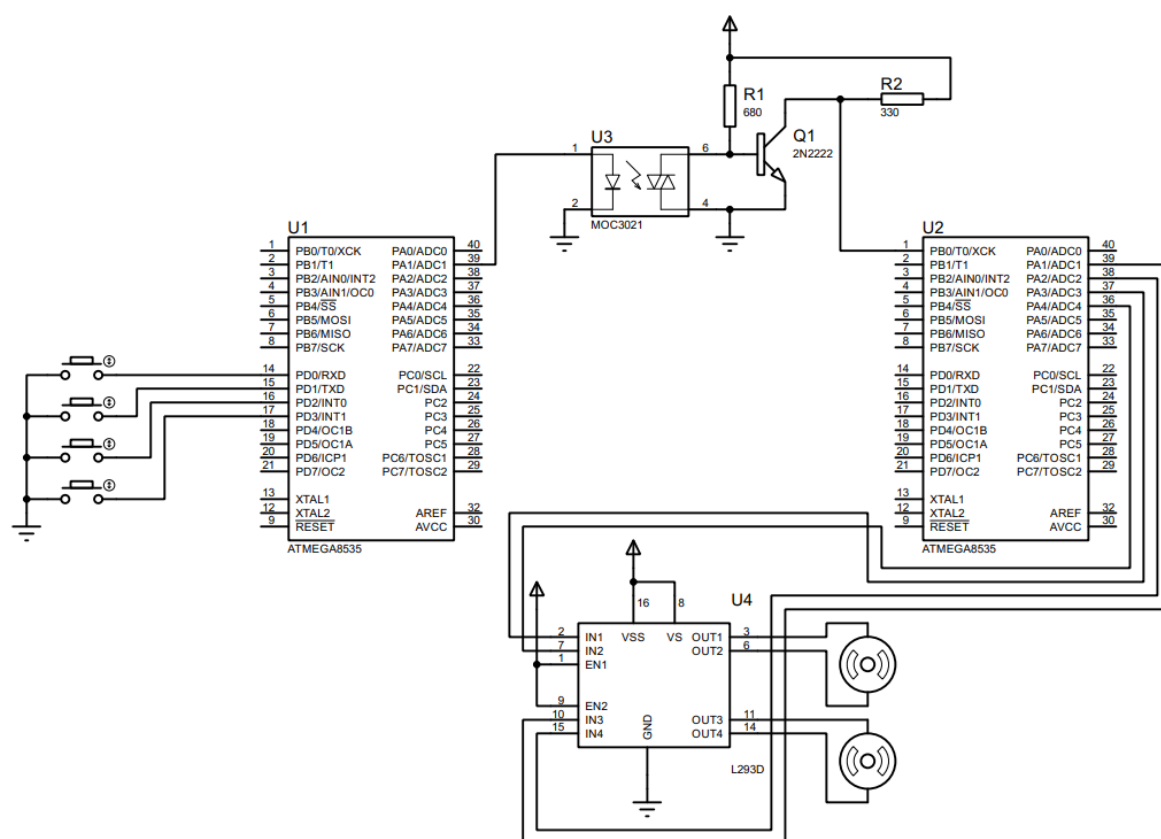


Imagen 1. Circuito creado en Proteus.

### Configuración previa al código

- **Transmisor**

De acuerdo con la imagen, el registro A se configura como de salida (PA1), mientras que el registro D como de entrada (PD0-PD3), activando las resistencias de pull-up.

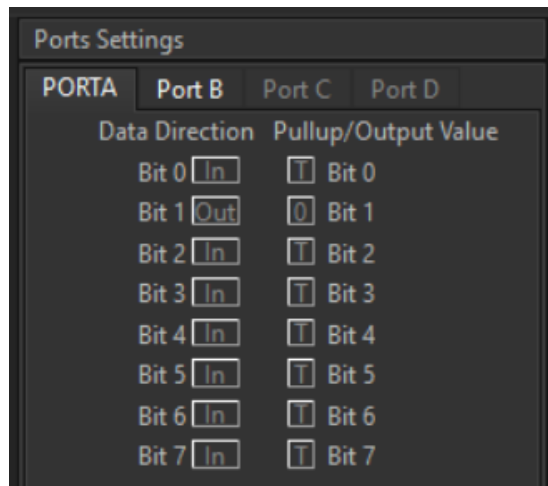


Imagen 2. Configuración del puerto A.

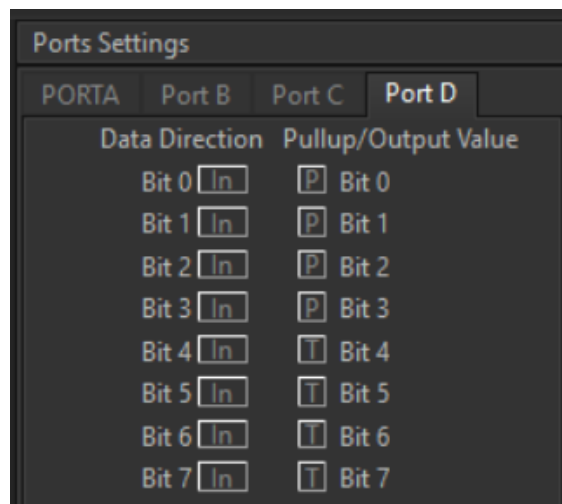


Imagen 3. Configuración del puerto D

- **Receptor**

De acuerdo con la imagen, el registro A se configura como de salida (PA1-PA4), mientras que el puerto B como de entrada (PB0) activando la resistencia de pull-up.

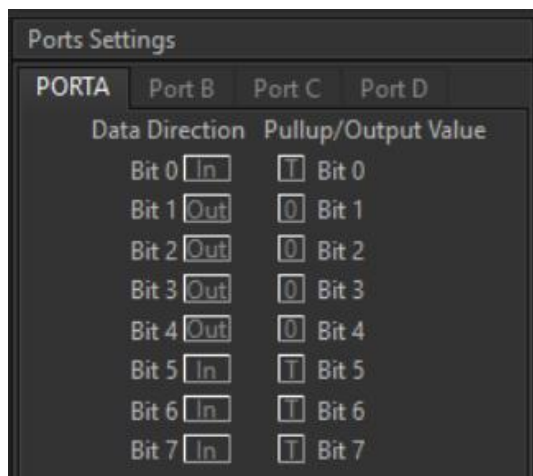


Imagen 4. Configuración del puerto A.

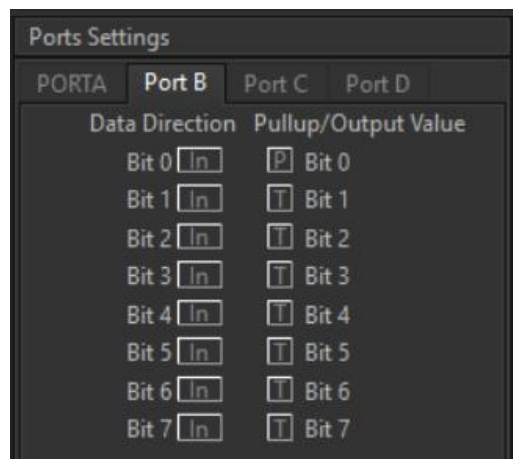


Imagen 5. Configuración del puerto B.

### Código del circuito de CodeVision:

- Transmisor

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4       Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Proyecto 1 'Transmisor'
8  Version : 1.0
9  Date    : 20/03/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26
27 #define IZQ PIND.0
28 #define DER PIND.1
29 #define ARR PIND.2
30 #define ABJ PIND.3
31

```

```

32 // Declare your global variables here
33
34 void main(void) {
35     // Declare your Local variables here
36
37     // Input/Output Ports initialization
38     // Port A initialization
39     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=Out Bit0=In
40     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (1<<DDA1) | (0<<DDA0);
41     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=0 Bit0=T
42     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
43
44     // Port B initialization
45     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
46     DDRB=(0<<ddb7) | (0<<ddb6) | (0<<ddb5) | (0<<ddb4) | (0<<ddb3) | (0<<ddb2) | (0<<ddb1) | (0<<ddb0);
47     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
48     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
49
50     // Port C initialization
51     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
52     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
53     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
54     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
55
56     // Port D initialization
57     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
58     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
59     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=P Bit2=P Bit1=P Bit0=P
60     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (1<<PORTD3) | (1<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);

```

```

62 // Timer/Counter 0 initialization
63 // Clock source: System Clock
64 // Clock value: Timer 0 Stopped
65 // Mode: Normal top=0xFF
66 // OC0 output: Disconnected
67 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
68 TCNT0=0x00;
69 OCR0=0x00;
70
71 // Timer/Counter 1 initialization
72 // Clock source: System Clock
73 // Clock value: Timer1 Stopped
74 // Mode: Normal top=0xFFFF
75 // OC1A output: Disconnected
76 // OC1B output: Disconnected
77 // Noise Canceler: Off
78 // Input Capture on Falling Edge
79 // Timer1 Overflow Interrupt: Off
80 // Input Capture Interrupt: Off
81 // Compare A Match Interrupt: Off
82 // Compare B Match Interrupt: Off
83 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
84 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
85 TCNT1H=0x00;
86 TCNT1L=0x00;
87 ICR1H=0x00;
88 ICR1L=0x00;
89 OCR1AH=0x00;
90 OCR1AL=0x00;
91 OCR1BH=0x00;
92 OCR1BL=0x00;

```

```

94 // Timer/Counter 2 initialization
95 // Clock source: System Clock
96 // Clock value: Timer2 Stopped
97 // Mode: Normal top=0xFF
98 // OC2 output: Disconnected
99 ASSR=0<<AS2;
100 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
101 TCNT2=0x00;
102 OCR2=0x00;
103
104 // Timer(s)/Counter(s) Interrupt(s) initialization
105 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
106
107 // External Interrupt(s) initialization
108 // INT0: Off
109 // INT1: Off
110 // INT2: Off
111 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
112 MCUCSR=(0<<ISC2);
113
114 // USART initialization
115 // USART disabled
116 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
117
118 // Analog Comparator initialization
119 // Analog Comparator: Off
120 // The Analog Comparator's positive input is
121 // connected to the AIN0 pin
122 // The Analog Comparator's negative input is
123 // connected to the AIN1 pin
124 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
125 SFIOR=(0<<ACME);

```

```
127 // ADC initialization
128 // ADC disabled
129 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
130
131 // SPI initialization
132 // SPI disabled
133 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
134
135 // TWI initialization
136 // TWI disabled
137 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);
```

```
139 while (1) {
140     // Place your code here
141     if (!IZQ) {
142         PORTA = 0xFF;
143         delay_ms(25);
144         PORTA = 0x00;
145         delay_ms(475);
146     } else if (!DER) {
147         PORTA = 0xFF;
148         delay_ms(50);
149         PORTA = 0x00;
150         delay_ms(450);
151     } else if (!ARR) {
152         PORTA = 0xFF;
153         delay_ms(75);
154         PORTA = 0x00;
155         delay_ms(425);
156     } else if (!ABJ) {
157         PORTA = 0xFF;
158         delay_ms(100);
159         PORTA = 0x00;
160         delay_ms(400);
161     }
162 }
163 }
```

- Receptor



```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4      Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Proyecto 1 'Receptor'
8  Version : 1.0
9  Date    : 20/03/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model    : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26
27 #define ARR 0x12 // 0001 0010
28 #define ABJ 0x0C // 0000 1100
29 #define IZQ 0x04 // 0000 0100
30 #define DER 0x02 // 0000 0010
31 #define ENTRADA PINB.0

```

```

33 // Declare your global variables here
34
35 void main(void) {
36     // Declare your local variables here
37     int contador = 0, i = 0;
38
39     // Input/Output Ports initialization
40     // Port A initialization
41     // Function: Bit7=In Bit6=In Bit5=In Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
42     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (1<<DDA4) | (1<<DDA3) | (1<<DDA2) | (1<<DDA1) | (0<<DDA0);
43     // State: Bit7=T Bit6=T Bit5=T Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=T
44     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
45
46     // Port B initialization
47     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
48     DDRB=(0<<ddb7) | (0<<ddb6) | (0<<ddb5) | (0<<ddb4) | (0<<ddb3) | (0<<ddb2) | (0<<ddb1) | (0<<ddb0);
49     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=P
50     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (1<<PORTB0);
51
52     // Port C initialization
53     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
54     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
55     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
56     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
57
58     // Port D initialization
59     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
60     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
61     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
62     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

```

```

64 // Timer/Counter 0 initialization
65 // Clock source: System Clock
66 // Clock value: Timer 0 Stopped
67 // Mode: Normal top=0xFF
68 // OC0 output: Disconnected
69 TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
70 TCNT0=0x00;
71 OCR0=0x00;
72
73 // Timer/Counter 1 initialization
74 // Clock source: System Clock
75 // Clock value: Timer1 Stopped
76 // Mode: Normal top=0xFFFF
77 // OC1A output: Disconnected
78 // OC1B output: Disconnected
79 // Noise Canceler: Off
80 // Input Capture on Falling Edge
81 // Timer1 Overflow Interrupt: Off
82 // Input Capture Interrupt: Off
83 // Compare A Match Interrupt: Off
84 // Compare B Match Interrupt: Off
85 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
86 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
87 TCNT1H=0x00;
88 TCNT1L=0x00;
89 ICR1H=0x00;
90 ICR1L=0x00;
91 OCR1AH=0x00;
92 OCR1AL=0x00;
93 OCR1BH=0x00;
94 OCR1BL=0x00;

```

```

96 // Timer/Counter 2 initialization
97 // Clock source: System Clock
98 // Clock value: Timer2 Stopped
99 // Mode: Normal top=0xFF
100 // OC2 output: Disconnected
101 ASSR=0<<AS2;
102 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
103 TCNT2=0x00;
104 OCR2=0x00;
105
106 // Timer(s)/Counter(s) Interrupt(s) initialization
107 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
108
109 // External Interrupt(s) initialization
110 // INT0: Off
111 // INT1: Off
112 // INT2: Off
113 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
114 MCUCSR=(0<<ISC2);
115
116 // USART initialization
117 // USART disabled
118 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCS22) | (0<<RXB8) | (0<<TXB8);
119
120 // Analog Comparator initialization
121 // Analog Comparator: Off
122 // The Analog Comparator's positive input is
123 // connected to the AIN0 pin
124 // The Analog Comparator's negative input is
125 // connected to the AIN1 pin
126 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
127 SFIOR=(0<<ACME);

```

```

129 // ADC initialization
130 // ADC disabled
131 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
132
133 // SPI initialization
134 // SPI disabled
135 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
136
137 // TWI initialization
138 // TWI disabled
139 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

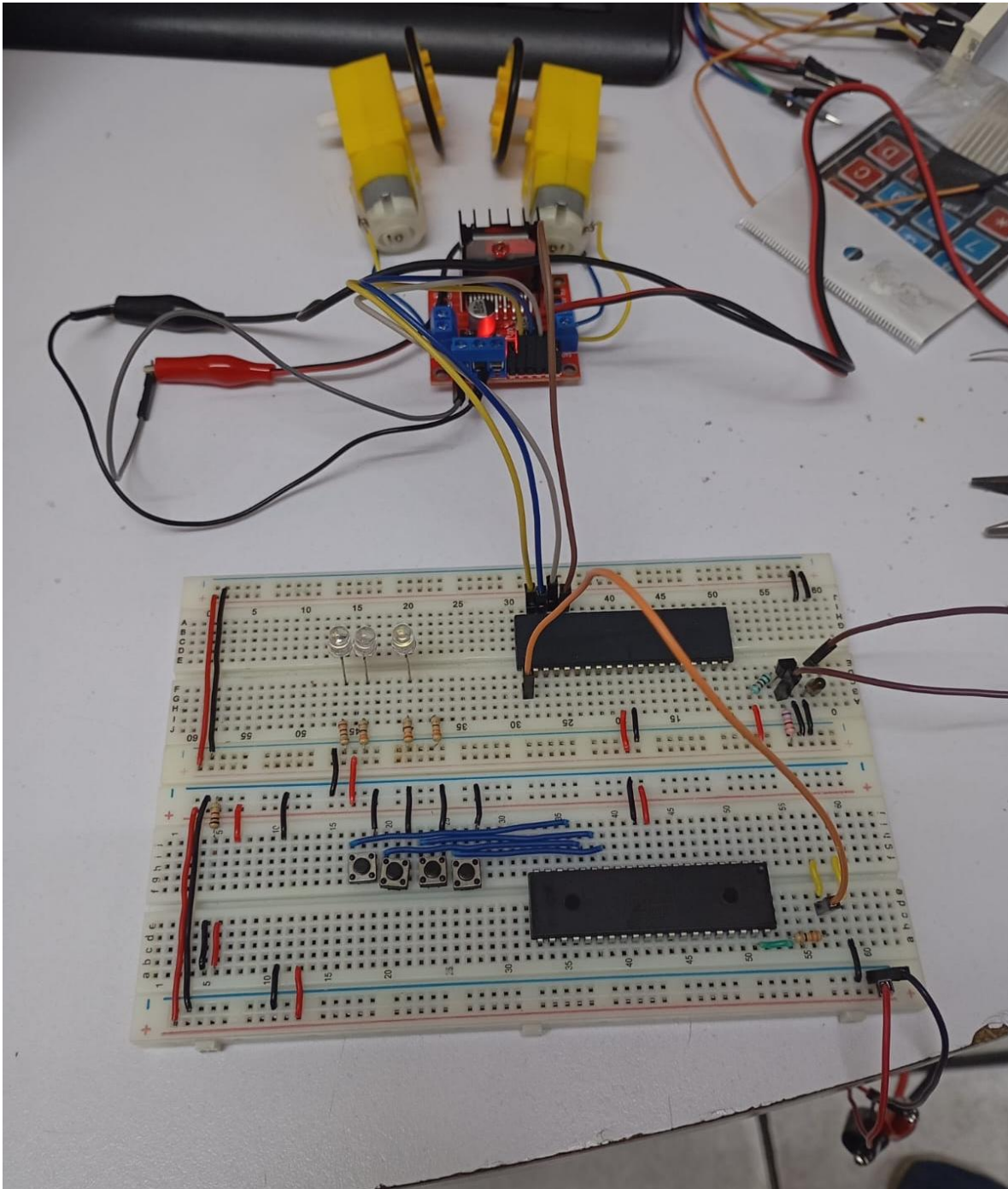
```

```

141 while (1) {
142     // Place your code here
143     contador = 0;
144
145     for (i = 0; i < 500; i++) {
146         if (ENTRADA) contador++;
147         delay_ms(1);
148     }
149
150     if (contador >= 15 && contador <= 35) {
151         PORTA = IZQ;
152     } else if (contador >= 40 && contador <= 60) {
153         PORTA = DER;
154     } else if (contador >= 65 && contador <= 85) {
155         PORTA = ARR;
156     } else if (contador >= 90 && contador <= 110) {
157         PORTA = ABJ;
158     } else {
159         PORTA = 0x00;
160     }
161 }
162 }

```

**Circuito físico**



## CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Para generar los pulsos provenientes del transmisor lo que se hace es jugar con los delays. En un principio se acordó que cada pulso debería de durar 500 ms y de ese tiempo asignar una parte al pulso mientras está en alto. El truco para generar el pulso para cada una las



direcciones del robot simplemente fue aumentar el tiempo en alto en cada una. Los tiempos fueron: 25 ms para la izquierda, 50 ms para la derecha, 75 ms para arriba y 100 ms para abajo.

En el receptor realmente no contamos la cantidad de pulsos sino más bien el tiempo que este permanece en alto. Para ello monitoreamos el estado del pulso 500 veces (porque cada pulso dura 500 ms), y cada 1ms verificamos que el pulso este en alto, si esto se cumple entonces aumentamos un contador. Después del conteo vemos si el valor entra en los distintos rangos que determinan la dirección a la que ira el robot, para la izquierda el tiempo debe estar entre 15 y 35 ms, para la derecha entre 40 y 60 ms, para arriba entre 65 y 85 ms y para abajo entre 90 y 110 ms, que de hecho empatan con los tiempos definidos en el transmisor solo que con una pequeña tolerancia para despreciar el ruido que pueda entrar.

Al final dentro de cada rango se activaron los bits correspondientes a la dirección detectada.

Por último, cabe destacar que en el código del transmisor se usa `else if` para detectar el restante de los botones, sin embargo, hay un problema con esto y es que, si se presionan dos botones a la vez, el único que va a funcionar es el de la izquierda puesto que se encuentra primero y por tanto tiene más prioridad. Para hacer que cada botón sea independiente lo mejor es usar `in if` para cada uno de ellos para que todos tengan la misma prioridad.

- **Meléndez Macedonio Rodrigo**

Para este primer proyecto se trabajó con 2 motores para simular el movimiento en 4 direcciones usando dos ruedas (izquierda, derecha, avanzar, retroceder), para ello se configuraron los motores para que reaccionen a los pulsos de los botones, cada uno debe reaccionar a diferentes pulsaciones y éstas deben entrar dentro de un rango específico de tiempo para ser detectadas correctamente. Para el movimiento a la izquierda se tiene un rango de 15-35 ms, para el movimiento a la derecha se tiene un rango de 40-60 ms, para que avance se tiene un rango de 65-85 ms y para que retroceda se tiene un rango de 90-110 ms. Es importante mencionar que para este proyecto no se toman en cuenta la cantidad de pulsos, si no el rango de tiempo que este permanece en alto.

Con estas configuraciones los motores cumplen su función y no hay problemas para que detecte el tipo de movimiento. Aunque al principio tuvimos un par de problemas para configurar los movimientos, pues algunos no reaccionaban y otros reaccionaban de una forma distinta a la esperada, sin embargo, logramos detectar el problema a tiempo y concluimos exitosamente el proyecto.

## BIBLIOGRAFÍA

- (s.f.). *Sensor Infrarrojo*. Wikipedia. Recuperado el 2 de abril de 2026, de [https://es.wikipedia.org/wiki/Sensor\\_infrarrojo](https://es.wikipedia.org/wiki/Sensor_infrarrojo)
- (s.f.). *Sensores Infrarrojos*. Luis Llamas. Recuperado el 2 de abril de 2026, de <https://www.luisllamas.es/>
- (s.f.). *Motorreductor*. Geek Electrónica. Recuperado el 2 de abril de 2026, de <https://geekelectronica.com/>